

This grade book application features a series of lab assignments developed by **Dr. Jim Pham** for **Ohlone College's CS-124 Data Structures** course.

The individual assignment is designed to strengthen technical proficiency and provide a comprehensive understanding of foundational data structures.

© Proprietary Material

Lab 1 - User Account Management System

Objective: Build a persistent, Object-Oriented C++ application to manage user credentials, tracking login/logout timestamps using custom class hierarchies and operator overloading. Your program should run continuously until the user selects the exit option.

Instructions:

Step 1: Data Hierarchy

You must implement a two-tier inheritance structure to handle date and time data.

1. Base Class: Time

- **Private Members:** int hour, int minute, int second.
- **Requirements:** Full set of mutators (setters), accessors (getters), a parameterized constructor, and a destructor.

2. Derived Class: DateTime (Publicly inherits from Time)

- **Private Members:** int month, int day, int year.
- **Requirements:** Full set of mutators/accessors, constructor, and destructor.
- **Formatting:** Must support display in Sun 02/25/2026 format and CSV storage in dd/mm/yyyy hh:mm:ss format.

Step 2: User Account Logic (Login Class)

This class acts as the data container for individual users and defines how they are compared.

- **Private Members:**
 - int userId
 - string username, string password
 - DateTime loginDateTime, DateTime logoutDateTime
- **Operator Overloading:** You must overload the following six relational operators to compare Login objects (typically based on their loginDateTime):
 - ==, !=, >, <, >=, <=

Step 3: Controller & Interface (Menu Classes)

This Part manages the "flow" of the program and file persistence.

1. Base Class: Menu

- Provides the structural foundation for the interface.

2. **Derived Class: LoginMenu**

- **Data Handling:** Must read from and save to users_data.csv.
- **Vector Management:** Use a `std::vector<Login>` users to hold account data in memory.
- **The Interface:** Implement a continuous loop displaying:
 1. **Login:** Prompt for credentials; allow **3 retries** for invalid attempts.
 2. **Create Account:** Add a new user to the vector.
 3. **Remove Account:** Delete an existing user.
 4. **Reset Password:** Update the password for a specific user.
 5. **Exit ('x'):** Trigger the `logout()` method to save all current data back to the CSV.

Step 4: Execution (main.cpp)

Your main function should be concise. It should instantiate the LoginMenu, trigger the initial data load, and start the program loop.

Submission Guidelines

- **File Naming:** lab1-YourName.zip
- **Contents:** All .h and .cpp files, the users_data.csv file, and screenshots of the running output.
- **Constraint:** This is an individual midterm assignment. Plagiarism will result in a zero.
- **Individual Work:** Concepts may be discussed, but the implementation must be your own original code.

Lab 2: Student Contact Management System

Objective: Continuing Lab 1 to develop an OOP system to manage student contacts using a custom-built generic list template. You will implement core Data Structure operations:

Accessing, Searching, Inserting, and Deleting.

Instructions:

Step 1: The Data Model (Person & Contact)

You will use inheritance to model the hierarchy of a contact.

- **Base Class: Person**
 - Store basic naming: first_name, middle_name, last_name.
- **Derived Class: Contact**
 - Inherits from Person.
 - **Private Data:** id, role, organization, address, city, county, state, zip, phone1, phone2, email.
 - **Requirements:** Implement all mutators, accessors, and **operator overloading** (specifically < and >) to allow the sorting algorithm to compare contact objects.

Step 2: The Data Structure (ArrayListType Template)

Instead of using a standard vector, you must build your own generic storage container.

- **Base Class Template: ArrayListType<T>**
 - **Members:** A pointer to an array of type T, length, and maxSize.
 - **Functionality:** Define methods for isEmpty(), isFull(), listSize(), maxListSize(), print(), isItemAtEqual(), insertAt(), insertEnd(), removeAt(), retrieveAt(), replaceAt(), and clearList().
 - **Sorting:** Implement a **Selection Sort** algorithm within this template to organize data.
- **Derived Class: ContactList**
 - Inherits from ArrayListType<Contact>.
 - This class specifically handles the loading/saving of contact_data.csv.

Step3: The Controller (ContactMenu)

This class manages the pointer to your list and drives the user interface.

- **Member Variable:** `ContactList* pList`; (Must use a pointer).
- **Menu Options:**
 1. **List of Contacts:** Display a formatted table using `<iomanip>`. Includes `doSortBy()` to sort by any field.
 2. **View a Contact:** Detail view of a single selected contact.
 3. **Add New Contact:** User input to create a new record.
 4. **Edit Contact:** Modify an existing record in the list.
 5. **Delete Contact:** Remove a record from the list.
 6. **Exit ('x'):** Save all changes back to `contact_data.csv` and terminate.

Step 4: Technical Requirements & Formatting

- **File Persistence:** All data must be read from and written to `contact_data.csv`.
- **Output Formatting:** Use `std::setw`, `std::left`, and `std::right` from the `<iomanip>` library to ensure the console output is professional and aligned.
- **Memory Management:** Since `pList` is a pointer, ensure proper allocation (new) and deallocation (delete) to avoid memory leaks.

Submission Checklist

- **Package:** `lab3-YourName.zip`
- **Contents:** All `.h` and `.cpp` files + output screenshots.
- **Individual Work:** Concepts may be discussed, but the implementation must be your own original code.

Lab 3: Enhancement to Lab 2 - Management System and Navigation

Objective: Continuing Lab 2 to develop an object-oriented programming system for managing user discussions using a doubly linked list. You will transition from contiguous, array-based storage to dynamic, node-based navigation.

Instructions:

Step 1: The Data Model (Discussion Class)

This class models a forum post. It must handle complex string parsing for multi-line text.

- **Inheritance:** Inherit from your existing Person class.
- **Data Members:**
 - string subject
 - DateTime postDate (Use your class from Lab 2)
 - string discussionText (Must handle the \ continuation character for multi-line inputs).
- **Key Requirement:** Implement **Operator Overloading** (<, >, ==, !=) to enable sorting and searching by subject, date, or author.

Step 2: The Data Structure (Generic Doubly Linked List)

Instead of a simple array, you will implement a **Template-based Doubly Linked List** to manage dynamic nodes.

- **The Node Structure:** Each node must contain data, a next pointer, and a back (previous) pointer.
- **Iterator Support:** Implement an Iterator class to navigate the list efficiently.
- **Core Navigation:**
 - moveFirst(): Jump to the head of the list.
 - moveLast(): Jump to the tail of the list.
 - moveNext() / movePrevious(): Step through records one by one.

Step 3: Algorithms: QuickSort & Binary Search

You are required to implement advanced algorithms that perform significantly better on large datasets.

- **QuickSort:** Implement a recursive QuickSort algorithm that can sort the Linked List by any field (subject, date, or name) in both ascending and descending order.
- **Binary Search:** Since Linked Lists do not support random access, your binary search logic must be optimized to find, edit, and delete specific discussions based on user input.

Step 4: The Interface (DiscussionMenu)

The menu must facilitate both management and navigation of the records.

- **Menu Options:**
 1. **List Discussions:** Display records with the ability to sort (QuickSort) and navigate (First/Next/Prev/Last).
 2. **View Details:** Use **Binary Search** to find a specific record and display its full content.
 3. **Add Discussion:** Insert a new node into the list.
 4. **Edit/Delete:** Use **Binary Search** to locate the record before modifying or removing the node.
 5. **Exit:** Serialize the data and save it to discussions.dat.

Step 5: Technical Specifications

- **File Persistence:** Read and write data using the discussions.dat format.
- **Formatting:** Use <iomanip> (setw, left, right) to create a clean, table-based terminal UI.
- **Memory Management:** Strictly follow C++ guidelines for new and delete to ensure no memory leaks occur during node deletion or list destruction.

Submission Checklist

- **Zip File:** lab4-YourName.zip
- **Content:** All header (.h) and implementation (.cpp) files, plus screenshots of successful Quicksort and Binary Search operations.
- **Individual Work:** Concepts may be discussed, but the implementation must be your own original code.

Lab 4: Enhancement to Lab 3 - Management System using Linked List

Objective: Continuing Lab 3 to develop an OOP system to manage user discussions using a **Doubly Linked List**. You will transition from a contiguous-array-based storage model to a dynamic, node-based navigation model.

Instructions:

Step 1: The Data Model (Discussion Class)

This class models a forum post. It must handle complex string parsing for multi-line text and inherit from your previous work.

- **Inheritance:** Inherit from your existing Person class (Lab 3).
- **Data Members:**
 - string subject
 - DateTime postDate (Use your class from Lab 2).
 - string discussionText (Must handle the \ continuation character for multi-line inputs).
- **Key Requirement:** Implement **Operator Overloading** (<, >, ==, !=) to enable sorting and searching by subject, date, or author.

Step 2: The Data Structure (Generic Doubly Linked List)

Instead of a simple array, you will implement a **Template-based Doubly Linked List** to manage dynamic nodes.

- **The Node Structure:** Each node must contain data, a next pointer, and a back (previous) pointer.
- **Iterator Support:** Implement an Iterator class to navigate the list efficiently.
- **Core Navigation:**
 - moveFirst(): Jump to the head of the list.
 - moveLast(): Jump to the tail of the list.
 - moveNext() / movePrevious(): Step through records one by one.

Step 3: Algorithms - QuickSort & Binary Search

You are required to implement advanced algorithms that perform significantly better on large datasets.

- **QuickSort:** Implement a recursive QuickSort algorithm that can sort the Linked List by any field (subject, date, or name) in both ascending and descending order.
- **Binary Search:** While typically used for arrays, you will apply binary searching logic (requiring mid-point navigation) to locate, edit, and delete specific discussions.

Step 4: The Interface (DiscussionMenu)

The menu must facilitate both management and navigation of the forum records.

- **Menu Options:**
 1. **List Discussions:** Display records with the ability to sort (QuickSort) and navigate (First/Next/Prev/Last).
 2. **View Details:** Use **Binary Search** to find a specific record and display its full content.
 3. **Add Discussion:** Insert a new node into the list.
 4. **Edit/Delete:** Use **Binary Search** to locate the record before modifying or removing the node.
 5. **Exit:** Serialize the data and save it to the binary file discussions.dat.

Step 5: Technical Specifications

- **File Persistence:** Read and write data using the discussions.dat format.
- **Formatting:** Use <iomanip> (setw, left, right) to create a clean, table-based terminal UI.
- **Memory Management:** Strictly follow C++ guidelines for new and delete to ensure no memory leaks occur during node deletion or list destruction.

Submission Checklist

- **Zip File:** lab4-YourName.zip
- **Content:** All header (.h) and implementation (.cpp) files, plus screenshots of successful Quicksort and Binary Search operations.
- **Integrity:** This is an individual assignment. You may discuss concepts with peers, but the final implementation must be your own original work.

Lab 5 – Enhancement Lab 4 using Map, Hashtable, and try-catch exception

Objective: Continuing Lab 4 to include Map, Hashtable, error handling, and associative data structures. You will transition from $O(n)$ or $O(\log n)$ searches to $O(1)$ constant-time lookups using a Hash Table.

Instructions:

Step 1: Defensive File I/O (Try-Catch Exceptions)

Instead of allowing the program to crash or fail silently if discussions.dat is missing or corrupted, you must implement a resilient file-loading sequence.

- Logic: Wrap your file stream (ifstream) and data parsing logic inside a try block.
- Custom Exception: Define a DiscussionException class to handle specific errors (e.g., File Not Found, Data Corruption).
- Retry Mechanism: If an exception is caught, alert the user and allow them to retry the operation or enter a different filename.

Step 2: Optimized Search (Hash Tables & Maps)

While your Linked List (Lab 4) is excellent for sequential navigation (Next/Previous), it is inefficient for searching. You will now implement associative containers to provide instant access.

- Custom Hash Table: Implement a Hash Table to store and retrieve Discussion objects. Use the provided hash-code logic:

$$h = \sum_{i=0}^{n-1} 31^{(n-1-i)} \cdot \text{str}[i]$$

- STL Map Integration: Use std::map to index discussions by a unique key (e.g., Subject or ID).
- Menu Updates: Modify doView, doEdit, and doDelete. These functions should now use the Hash Table or Map to find records instantly rather than iterating through the Linked List.

Step 3: The "Changelog" Requirement

Because Lab 5 is an "open" improvement, your submission must include a precise list of changes. You must document:

1. The exact locations of your Try-Catch blocks.
2. Which menu features now utilize the Hash Table for $O(1)$ lookup.

3. How you maintained synchronization between the LinkedList (for navigation) and the Hash Table (for searching).

Step 4: Integrated Architecture

Your system now operates with three distinct layers for data management:

- LinkedList: Used for "Move Next," "Move Previous," and "First/Last" navigation.
- Hash Table / Map: Used for "Find," "View Details," and "Edit/Delete."
- Exception Handler: Wraps the File I/O to ensure system stability.

Step 5: Technical Specifications

- Formatting: Maintain professional table output using `<iomanip>`.
- Collision Handling: Ensure your custom Hash Table accounts for collisions (e.g., using Linear Probing or Chaining).
- Persistence: Ensure that changes made in the Hash Table are reflected back in the Linked List before saving to `discussions.dat`.

Submission Checklist

- Package: `lab5-YourName.zip`
- Content: All `.h` and `.cpp` files, screenshots of the retry-logic after a file error, and the speed-optimized search results.
- Individual Work: Concepts may be discussed, but the implementation must be your own original code.

Lab 6: Enhancement to Lab 5 - Grade Book App using Binary Tree

Objective: Continuing Lab 5 to develop an OOP system that uses a **Tree Structure** to manage and calculate course grades. You will transition from linear lists to a parent-child hierarchy representing the course, its categories, and individual assignments.

Instructions:

Step 1: The Data Structure (Binary Tree Template)

You must implement a generic **Binary Tree** to represent a hierarchical structure. While a course folder system typically has multiple children per folder, you will implement this using the provided binary tree interface.

- **The Root Node:** Fall 2023 CS-124-03.
 - **The Branches (Group IDs):** Participation, Quizzes, Labs, Midterm, and Final Exam.
 - **The Leaves (Data):** Individual assignments linked to their respective categories.
-

Step 2: Weighted Grade Logic

Your program must calculate the weighted final grade based on the following Fall 2023 syllabus rules:

Group Weights:

- **Class Participation:** 10%
- **Quizzes:** 10%
- **Labs:** 30%
- **Midterm:** 20%
- **Final Exam:** 30%

Calculation Formula:

1. **Category Score:**
$$\left(\frac{\text{Total Earned Points}}{\text{Total Possible Points}} \right) \times 100$$
 2. **Weighted Total:**
$$\sum (\text{Category Score} \times \text{Weight \%})$$
 3. **Final Result:** Divide the weighted total by the sum of weights (100).
-

Step 3: Class Specifications

Class: Assignment

- **Data Members:** Id, GroupId, Name, StartDate, EndDate, PossiblePoints, Points.
- **Requirements:** Must be compatible with the Tree interface and support CSV serialization.

Class: AssignmentMenu

You must implement the following controller methods:

- **doList:** Perform a tree traversal to display all course data hierarchically.
 - **doAdd / doRemove:** Insert or delete nodes from the tree structure.
 - **doEdit:** Locate a node and modify assignment fields.
 - **calculateGrade:** Traverse the tree to aggregate points per category and output the final letter grade (A–F) using <iomanip>.
 - **doSave:** Export the current tree data back to assignments.csv.
-

Step 4: Technical Requirements

- **Templates:** Use the provided tree.h and binary_tree.h templates. Ensure your Assignment class works seamlessly with template <typename T>.
 - **File I/O:** Initialize the tree using FA2023-CS-124-03-grade-scale.csv (for the structure) and assignment_data.csv (for the records).
 - **Formatting:** Use std::setw and std::fixed to ensure the grade report is aligned and professional.
-

Submission Checklist

- **Package:** lab6-YourName.zip
- **Contents:** All .h and .cpp files + Screenshots of the Tree display and the final grade calculation.
- **Individual Work:** This is a summative demonstration of your understanding of Trees. The implementation must be your own.